

Week 2

Superstore Sales Dashboard

Data Modeling, DAX & Sales Dashboard

J.MOHANA SRI

E25DA027

B.Sc Data Science and AI

Table of Contents

1.Introduction

2. Project Overview

2.1 Dataset

2.2 Problem Statement

2.3 Objectives

2.4 Tasks & Deliverables

3. Data Modeling Approach

4. Building the DAX Measures

4.1 Setting Up the Dedicated Measures Table

4.2 Creating the Table_Measures Table

4.3 Average Sales Measure

4.4 Profit Margin % Measure

4.5 Total Order Measure

4.6 Total Profit Measure

4.7 Total Quantity Measure

4.8 Total Sales Measure

4.9 Completed Measure Table

4.10 Measure Summary Table

5. Final Interactive Dashboard

5.1 Dashboard Contents

1. Introduction

This report documents the second stage of the Superstore Sales Dashboard project, following on from the data cleaning and preparation work completed in Week 1. Having established a reliable, analysis-ready version of the Superstore Sales Dataset, the focus of this stage shifts to data modeling and calculation logic: organizing the tables into a coherent model, writing the DAX measures that management needs, and using those measures to assemble a working, interactive dashboard in Power BI.

The driving business need behind this work is straightforward. Management wants a single place to monitor sales performance, profitability, order volume, and regional trends without waiting on ad-hoc spreadsheet requests. An interactive Power BI dashboard, backed by a clean data model and a well-organized set of measures, is the intended solution.

This report is structured to walk through that process in the order it was carried out: first the project brief and objectives, then the data modeling approach, then a step-by-step account of each DAX measure as it was built, and finally the assembled dashboard itself along with a summary of what was delivered.

2. Project Overview

2.1 Dataset

The dashboard is built on the Superstore Sales Dataset, which was preprocessed during Week 1 of the project. That preprocessing stage handled the initial cleaning: removing duplicate records, correcting data types, and standardizing category and region naming so the dataset would behave consistently once loaded into Power BI's data model.

2.2 Problem Statement

Management wants an interactive dashboard to monitor sales, profit, orders, customers, and regional performance. Rather than static exports, the dashboard needs to support drill-down, cross-filtering, and at-a-glance KPI monitoring so that different stakeholders can explore the same underlying data from their own angle — a regional manager filtering to their territory, or an executive scanning the top-line numbers.

2.3 Objectives

- Establish table relationships and data model structures (Star Schema).

- Write foundational DAX measures: Total Sales, Total Profit, Total Orders, Total Quantity, Average Sales, and Profit Margin %.
- Construct visual layouts using KPI Cards, Stacked Bar Charts, Line Charts, and Decomposition Trees.

2.4 Tasks & Deliverables

- Build a dedicated DAX measure table (_Measures).
- Create a top-line executive summary bar with key KPI cards.
- Build category performance and regional profitability visual breakdowns.
- Deliver an interactive Superstore Sales Dashboard.

3. Data Modeling Approach

Before any measures were written, the underlying data model needed a clear structure. The `samplesuperstore` table holds the transactional data — one row per order line, with fields such as Order ID, Sales, Profit, Quantity, Category, Sub-Category, Region, and Order Date. Rather than writing every calculation directly against these raw columns scattered through report visuals, a separate calculations layer was introduced.

A new table, `Table_Measures`, was added purely to hold DAX measures. It carries no imported rows of its own; its single placeholder column exists only so the table appears in the model. Every KPI used on the dashboard — Total Sales, Total Profit, Total Order, Total Quantity, Average Sales, and Profit Margin % — lives inside this one table. This mirrors a common Power BI convention (sometimes called a "measures table" or "_Measures" table) that keeps business logic visually and organizationally separate from source data, which makes the model easier to audit, document, and hand off to other developers.

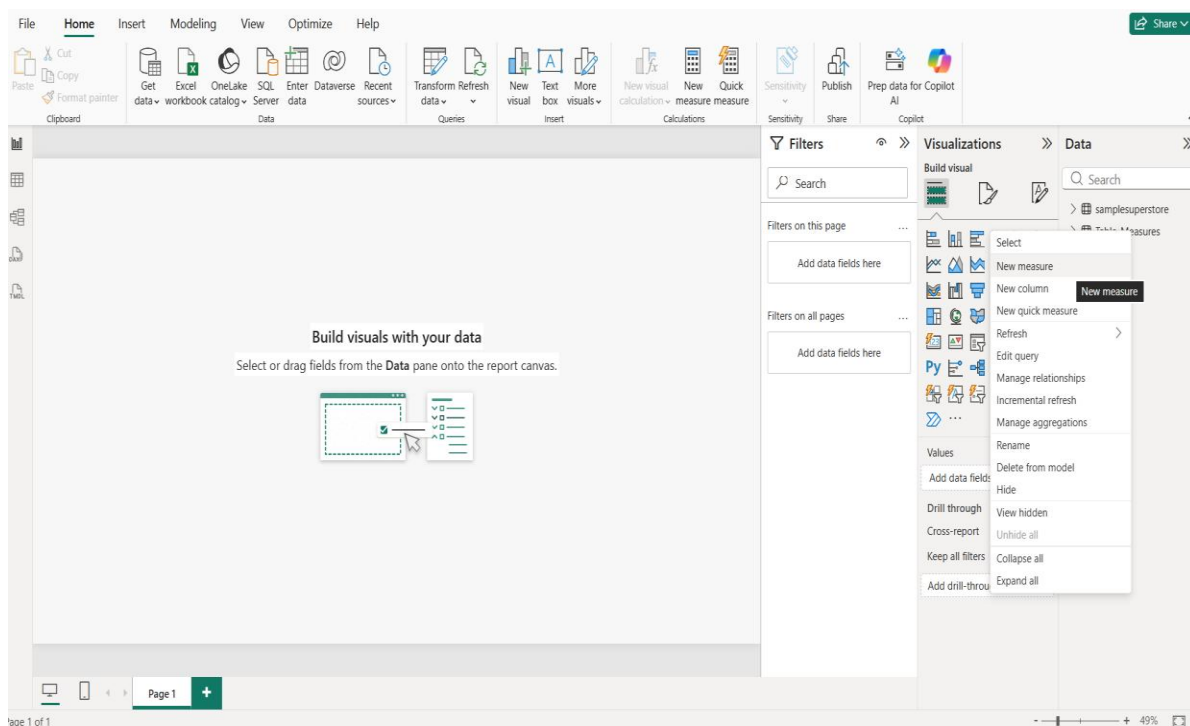
With that structure in place, the following sections walk through each measure as it was built, in the order shown by the screenshots captured during development.

4. Building the DAX Measures

The measures table was built step by step in Power BI Desktop, as documented below. Each step shows the relevant screen at the time the measure was written, followed by a short explanation of the formula and its role in the final dashboard.

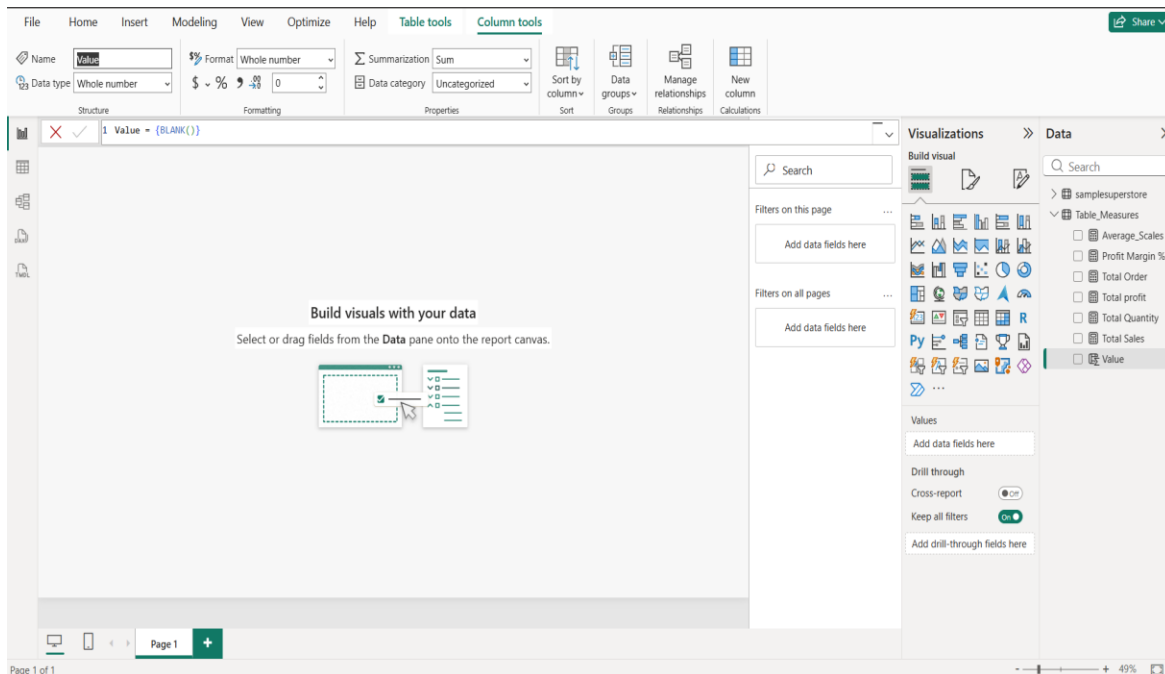
4.1 Setting Up the Dedicated Measures Table

Work began on a blank report canvas connected to the samplesuperstore dataset. A right-click on the Data pane opened the context menu used to add a new table, the first move toward building a dedicated DAX measure table. Keeping every calculation in its own table, rather than scattering measures across the raw data tables, makes the model easier to navigate, document, and maintain as the number of measures grows.



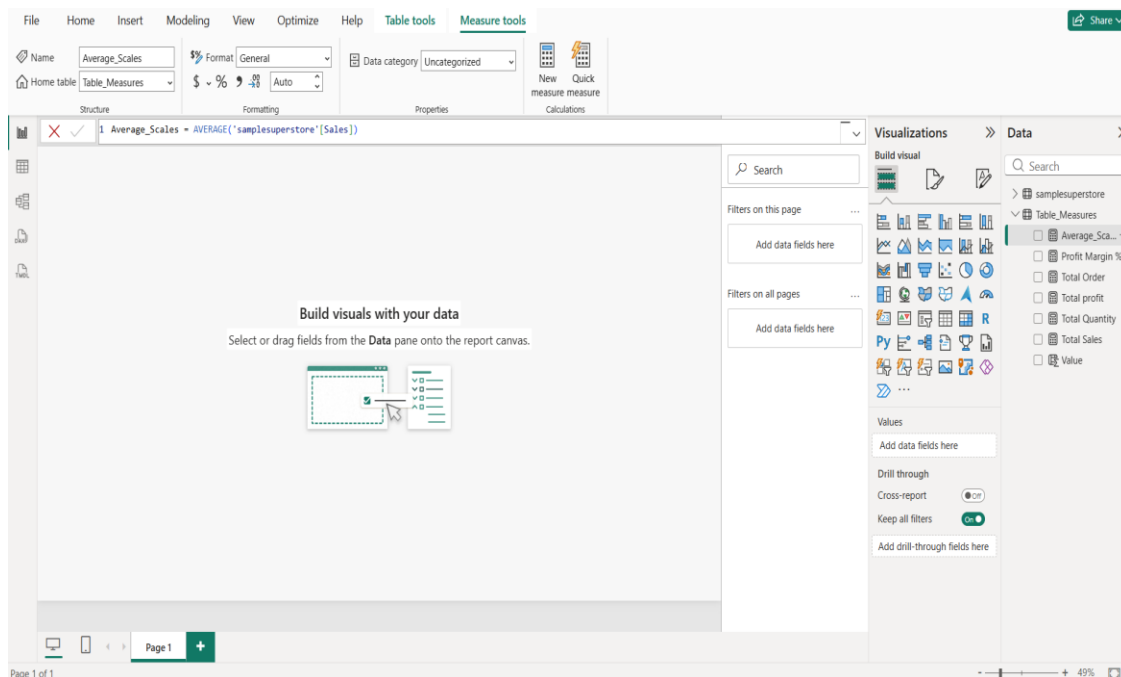
4.2 Creating the Table_Measures Table

A new table named Table_Measures was created to house all DAX measures for the report. A placeholder column, simply named "Value" and set to {BLANK()}, was added so the table would exist in the model; this column carries no data of its own and exists only as a home for the measures that follow. The table now appears in the Data pane alongside samplesuperstore, clearly separated from the imported fields.



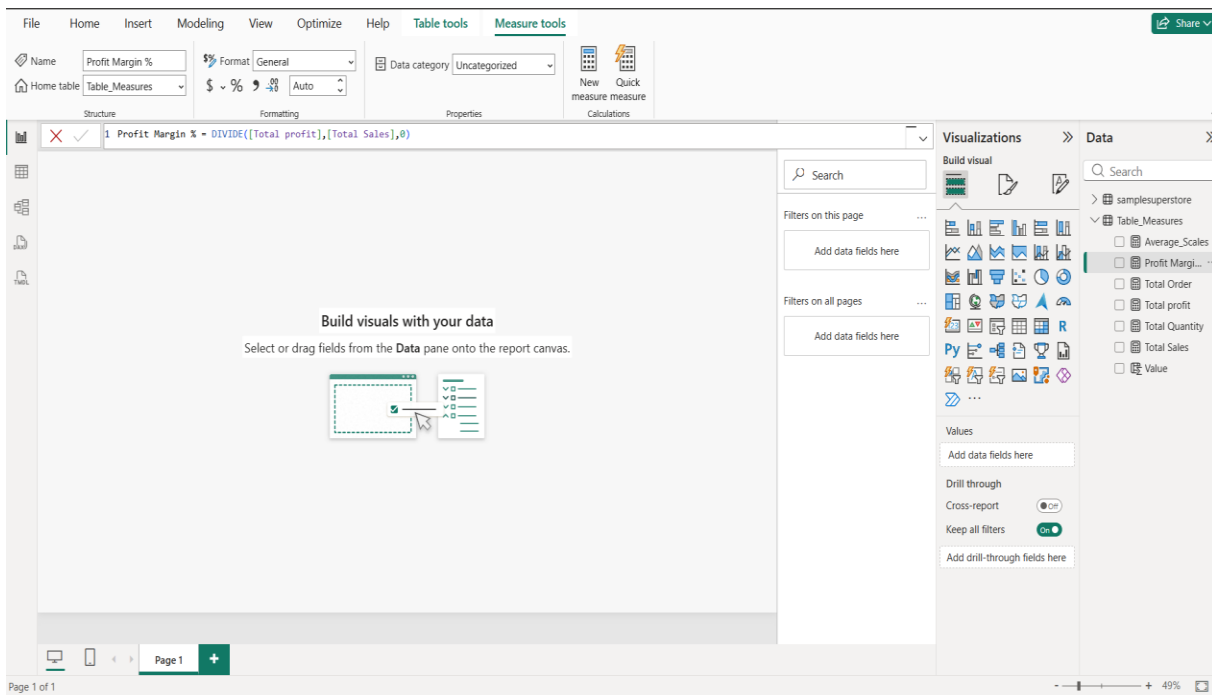
4.3 Average Sales Measure

The first real measure, `Average_Scales`, was written as `AVERAGE('samplesuperstore'[Sales])`. This returns the mean sale value across all transactions in the dataset and gives management a quick sense of typical order size, independent of overall volume.



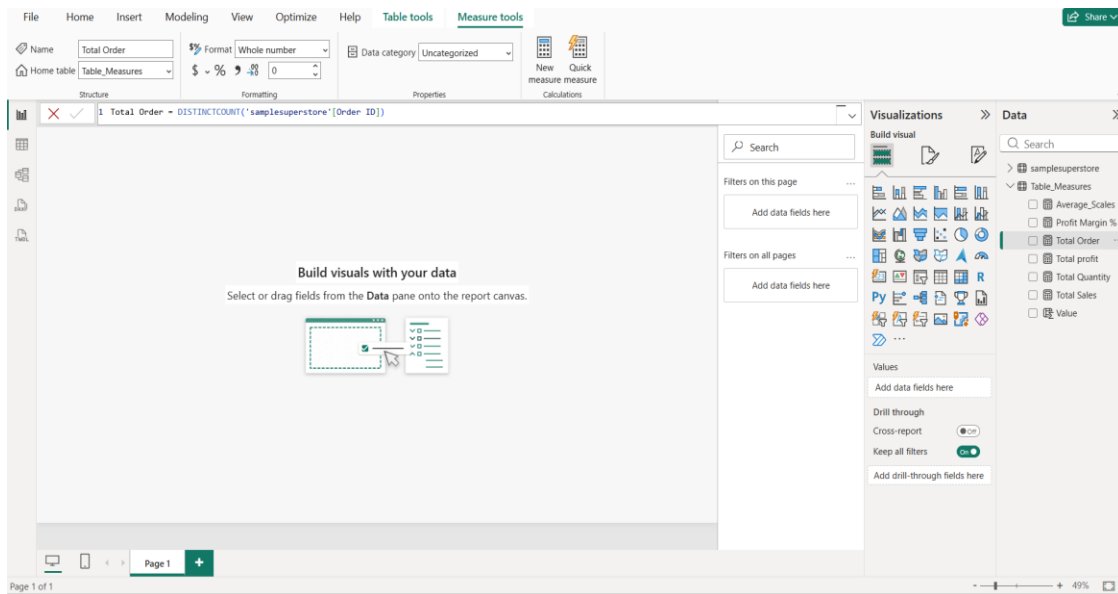
4.4 Profit Margin % Measure

The Profit Margin % measure was defined as `DIVIDE([Total profit],[Total Sales],0)`. Using `DIVIDE` rather than the plain `/` operator is a standard DAX safeguard: it returns the specified fallback value (0) instead of an error whenever Total Sales is zero, which keeps the measure stable across every filter context the dashboard can produce.



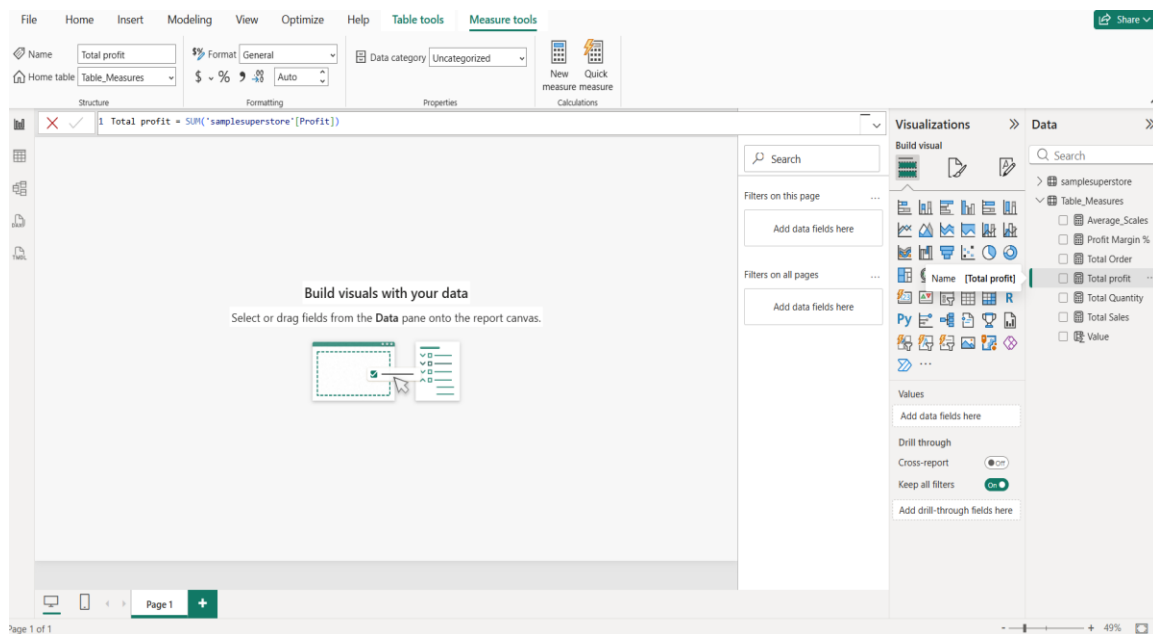
4.5 Total Order Measure

Total Order was built using `DISTINCTCOUNT('samplesuperstore'[Order ID])`. Because a single order can contain several line items, a plain row count would overstate the number of orders; `DISTINCTCOUNT` instead counts each unique Order ID once, giving an accurate order total.



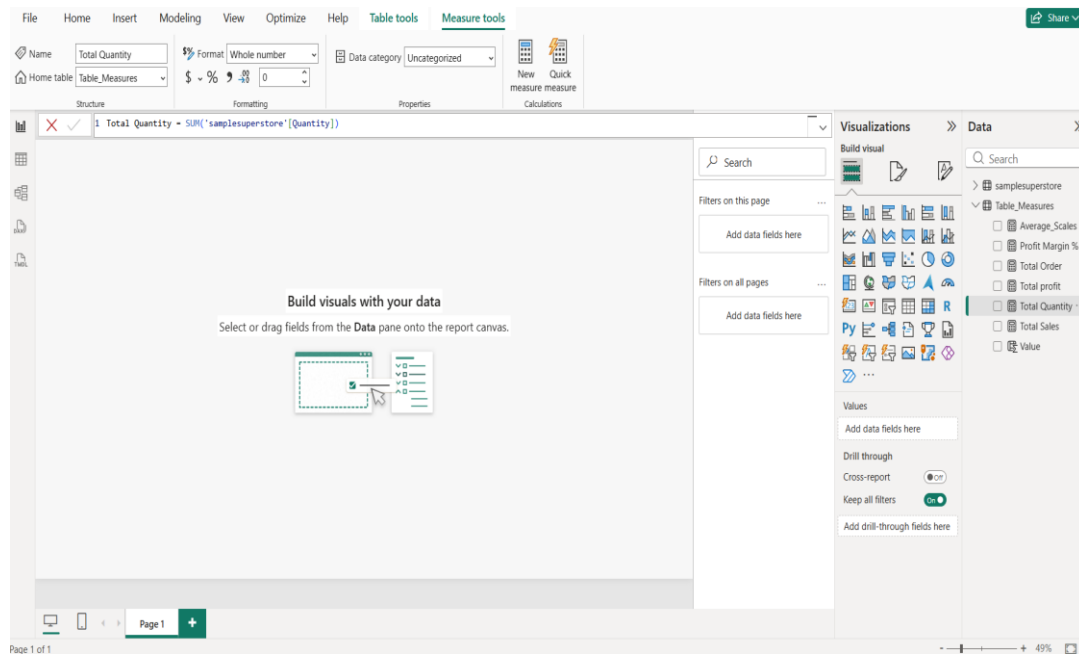
4.6 Total Profit Measure

Total profit was calculated with `SUM('samplestore'[Profit])`, aggregating the Profit column across whatever rows are in context. This measure underpins the Profit Margin % calculation and the Total profit by Sub-Category breakdown used later in the dashboard.



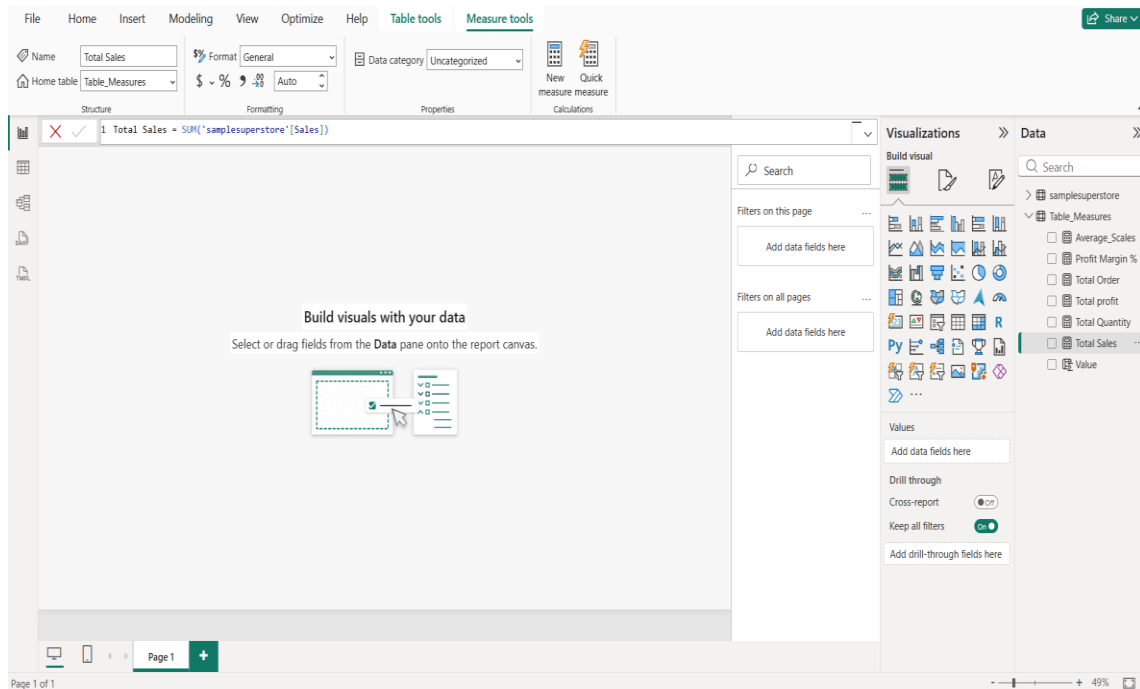
4.7 Total Quantity Measure

Total Quantity was defined as `SUM('samplesuperstore'[Quantity])`, giving the total number of units sold. This measure feeds the Total Quantity KPI card on the executive summary bar of the finished dashboard.



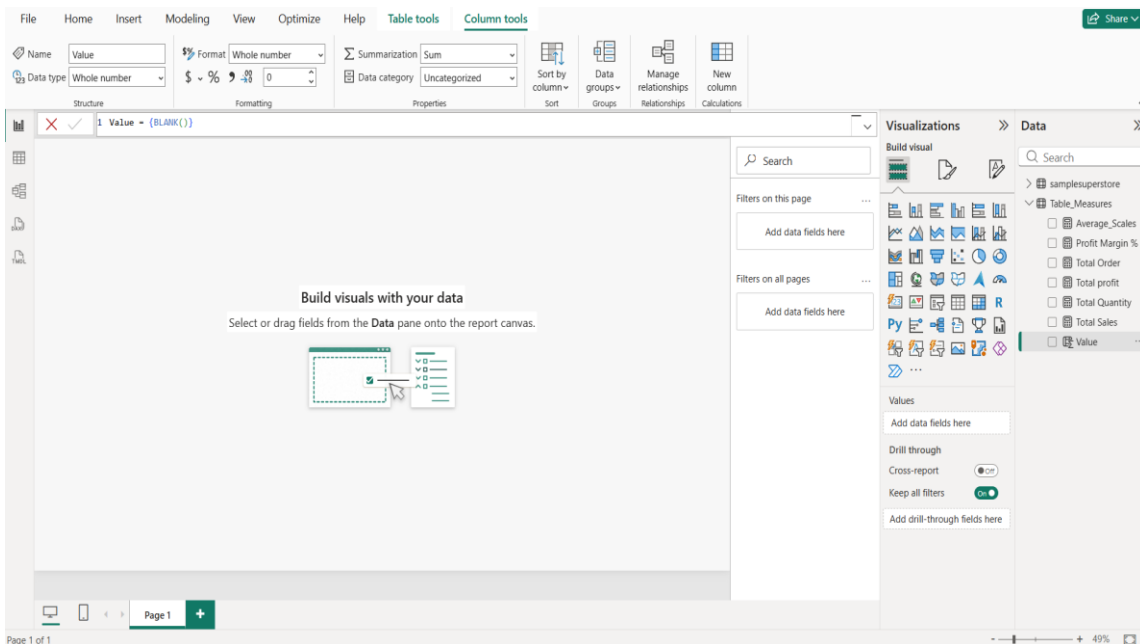
4.8 Total Sales Measure

Total Sales was created using `SUM('samplesuperstore'[Sales])`, completing the core revenue measure used throughout the report. It is referenced directly by several visuals (Total Sales by Region, by Category, and by Order ID) and indirectly by Profit Margin %.



4.9 Completed Measure Table

With all six foundational measures in place — Average_Scales, Profit Margin %, Total Order, Total profit, Total Quantity, and Total Sales — the Table_Measures table was complete. Every measure needed for the KPI cards, bar charts, donut chart, line chart, and gauge on the final dashboard now lived in one clearly labelled table, ready to be dragged onto the canvas.



4.10 Measure Summary Table

The table below summarizes all six foundational measures built for the dashboard, alongside their DAX formulas and business purpose.

Measure Name	DAX Formula	Purpose
Total Sales	SUM('samplesuperstore'[Sales])	Total revenue across all orders
Total profit	SUM('samplesuperstore'[Profit])	Total profit across all orders
Total Order	DISTINCTCOUNT('samplesuperstore'[Order ID])	Count of unique orders placed
Total Quantity	SUM('samplesuperstore'[Quantity])	Total units sold
Average_Scales	AVERAGE('samplesuperstore'[Sales])	Average sale value per line item
Profit Margin %	DIVIDE([Total profit],[Total Sales],0)	Profitability as a percentage of sales

5. Final Interactive Dashboard

With all measures in place, the visuals were assembled into a single-page executive dashboard. The layout groups a top row of KPI cards and category/region breakdowns, with supporting charts and a slicer beneath, so a viewer's eye moves naturally from the highest-level numbers down to the detail behind them.

5.1 Dashboard Contents

- KPI cards for Total Quantity, Profit Margin %, and Total Sales.
- A horizontal bar chart of Total Profit by Sub-Category.
- A horizontal bar chart of Total Sales by Region.
- A donut chart of Total Sales by Category.
- A line chart of Total Sales by Order ID.
- A gauge showing Total Sales split by Segment.
- An Order Date slicer enabling cross-filtering across every visual on the page.

